calBERT: Securing Routers Serving IoT Networks using Contrastive Augmented Learning

John Carter
Department of Computer Science,
Drexel University,
Philadelphia, PA, USA
jmc683@drexel.edu

Spiros Mancoridis
Department of Computer Science,
Drexel University,
Philadelphia, PA, USA
mancors@drexel.edu

Pavlos Protopapas
School of Engineering and Applied Sciences,
Harvard University,
Cambridge, MA, USA
pavlos@seas.harvard.edu

Brian Mitchell
Department of Computer Science,
Drexel University,
Philadelphia, PA, USA
bmitchell@drexel.edu

Benji Lilley
Department of Computer Science,
Drexel University,
Philadelphia, PA, USA
bl857@drexel.edu

Abstract

Previous work on home router security has shown that using system calls to train a transformer-based language model built on a BERT-style encoder using contrastive learning is effective in detecting several types of malware, but the performance remains limited at low false positive rates. In this work, we demonstrate that using a high-fidelity eBPF-based system call sensor, together with the novel introduction of contrastive augmented learning (which introduces controlled mutations of negative samples), improves detection performance, especially at low false positive rates. For example, for one malware pattern described below, detection at a 0.5% target false positive rate increases from 0% in the previous work to 70.56% using our contrastive augmented learning approach. However, for some of the system call results, such as for the stealthy advanced persistent threat (APT) malware, the detection could be improved at the lowest false positive rates examined. To this end, we introduce a novel network packet abstraction language that enables the creation of a pipeline similar to the system call data and show that the introduction of network behavior yields improved performance for network-focused malware at low false positive rates. In more than one case, detection at the lowest false positive rate improves from 0.00% using system calls to 100.00% using network traffic. Lastly, we implement these methods in an online router anomaly detection framework to validate the approach in an Internet of Things (IoT) deployment environment.

CCS Concepts

• **Security and privacy → Malware and its mitigation**.

Keywords

anomaly detection, BERT, contrastive augmented learning, router security, network security, edge security

ACM Reference Format:
John Carter, Spiros Mancoridis, Pavlos Protopapas, Brian Mitchell, and Benji Lilley. 2026. calBERT: Securing Routers Serving IoT Networks using Contrastive Augmented Learning. In *Proceedings of the 6th ACM Workshop on Secure and Trustworthy Cyber-Physical Systems (SaT-CPS '26), June 23–25, 2026, Frankfurt am Main, Germany*. ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3806008.3811704

SaT-CPS '26, Frankfurt am Main, Germany

ACM ISBN 979-8-4007-2610-1/2026/06
https://doi.org/10.1145/3806008.3811704

1 Introduction

Home routers serve as an important component in any home network as they are the gateway to the outside world. Nevertheless, home network security is often overlooked by security analysts despite the rise of connected and unmonitored Internet of Things (IoT) devices. The existence of one or more non-human-monitored devices connected to the network means that there must be a central location to aggregate the health of the network. As a result, it becomes clear that routers are the best location to do this type of network inference, since all network traffic, and by extension obvious signs of infection, must pass through the router.

Previous work by the authors [2] demonstrates that the use of a specialized language called **sys2vec** for the embedding of calls to the Linux kernel-level operating system (OS) is effective for protecting home routers by using unsupervised anomaly-detection. However, the approach described in that work showed limited robustness at low false positive rates, which is a critical operating regime for always-on router deployments.

This work replicates the environment and the malware in that research to illustrate three extensions to that approach. The first is the introduction of *contrastive augmented* learning to the BERT-style language-model training process, where controlled mutation of negative samples improves robustness at low false positive rates compared to using contrastive learning alone. The second is the introduction of a simple network packet abstraction language that enables us to apply a similar pipeline to that of the system calls and capture complementary network-level behaviors, which leads to improved detection for network-focused malware at low false

positive rates. We find that for the best detection, both a system call detector and a network packet detector should be deployed concurrently. The last contribution is an online anomaly detection framework using the methodology described in this work, which illustrates how the approach can operate in a practical deployment.

The system calls and network packets are collected during periods of strictly benign behavior, as well as separately during periods of malware infection. The packet data include all traffic on its network. In order to isolate malicious behavior, rather than using a mixture of benign and malicious behavior, only system calls and packets with process identifiers (PIDs) matching those generated by the malware during their execution are considered during evaluation. For deployment, where PID filtering is not available, an exponential moving average (EMA) is used to smooth the continuous stream of observations, allowing the detector to operate without requiring PID filtering while reducing false positives.

The system call data are transformed using sys2vec, as described in our previous work [2]. In contrast, the packet data are first generalized using a simple packet abstraction language. To generalize, we select a portion of the available packet header fields, such as specific IP addresses and port numbers, and put them into abstract categories before concatenating them together. Following the abstraction procedure, each distinct packet abstraction seen in the benign training data is embedded into a 64-dimensional space using net2vec, which provides the same form of embeddings as sys2vec but is used for our packet language.

The packet data are split into 70-word sentences in which each packet contains eight tokens, with a window length empirically selected to balance temporal context and vocabulary coverage. Anchor samples, positive samples, and negative samples were created from these sentences to feed into our contrastive augmented BERT-style model framework. The choice of these samples is crucial for model training, as it attempts to bring the positive sample closer while simultaneously pushing the negative sample further away from the anchor during the learning process [20]. The triplet selection algorithm is shown in more detail in Figure 2 and is explained in more detail below. The randomly chosen negative sample is mutated prior to being passed into the model by replacing a subset of the tokens in the sentence with randomly selected packets observed in the training dataset. We also evaluated contrastive learning without mutation, as in [2], but found that adding controlled perturbations to negative examples improved training robustness for the system call-based pipeline, as summarized in Table 2. For the network-based pipeline, the results show that mutation is not necessary for a robust detector.

In the results section, we evaluate how well the methods presented in this study work in two scenarios: offline and online. The offline analysis mirrors many in the literature in which data are collected offline and evaluated later using machine learning algorithms. This type of model inference is extremely useful for advancing the state-of-the-art since it is a controlled environment to run the code and draw conclusions. However, offline analysis can be lacking in real-world applicability in this domain since this type of analysis would never be used in a production setting. To this end, we also provide an online analysis section in which we collect the data on the router, send the data to another device on the network for model inference (known as a fog architecture), and send the results to the cloud for user notification. The reason for sending the data to another device on the local area network (LAN) is to ensure that the model inference does not degrade the router performance. This additional analysis provides the reader with invaluable information regarding the feasibility of such a method in the wild, and therefore presents a more complete research contribution.

The paper is organized as follows: First, it describes related work in Section 2. Following this, the methodology and setup are explained in Section 3. This includes the network ecosystem that the home router serves and the malware patterns that were created according to the specifications described in our previous work [2]. Section 3 then examines in detail how we collect the network packets and transform them to be used by our language-model framework, and summarizes how the system calls are transformed since this was explained in detail in prior work. Afterward, it describes how both contrastive augmented language models are trained and evaluated, which is a very similar process for both system calls and network packets. Next, the online anomaly detection framework is described, which provides a deployable version of the research presented. After describing the setup, the results are presented in Section 4. Lastly, it identifies avenues for future work in Section 5 and provides a list of research contributions in Section 6.

2 Related Work

2.1 Behavioral Malware Detection

Behavioral malware detection emerged several years ago as a response to the rise of zero-day and obfuscated malware, which has made traditional signature matching anti-virus protection less effective [15]. The reason for this is because antivirus detection relies on having seen malware that matches the binary signatures found in its database. However, malware obfuscation techniques work to change and/or encrypt their code with each replication, like polymorphic and metamorphic malware [14].

Behavioral malware detection seeks to model the benign behavior of a device in order to flag behavior deviating from this benign model as malicious. Training requires only benign data while malware traces are used solely for evaluation. The benefit of this approach is that no prior knowledge of malware is necessary and only benign data are necessary for model training, which is often readily available, in contrast to realistic and usable malware data. Several shallow machine learning models have been used for anomaly detection, such as one-class SVMs, autoencoders, and random forests [1] [13]. More recently, sequence-based approaches have been explored to capture temporal structure in device behavior, motivating the use of transformer-style models in subsequent sections of this work.

Although there has been work in the area of online anomaly detection, the area has been much less explored than traditional offline anomaly detection using static datasets. There exists an online anomaly detection study that explores the effectiveness of a framework for testing the quality of data streamed in a large telecommunication system [19]. This is relevant to our work in terms of the use of data streams, but differs in its application as it does not pertain to malware detection. Similarly, another study examines online anomaly detection for sensor systems and highlights their requirements such as accuracy, robustness, resource efficiency, and

performance [28]. These studies help frame the broader landscape of online detection, but they do not address the specific challenges posed by router-level behavioral monitoring, which motivates the online component of our work.

2.2 Large Language Models for Malware Detection

The popularity of large language models (LLMs) has grown significantly as commercial chatbot applications are used for tasks such as writing and coding. Although there has been an extensive amount of research into training these models using natural spoken languages, there has been comparatively less exploration of applying LLM-style sequence modeling to specialized machine-generated languages such as system calls or packet abstractions.

Significant research exists to harness the power of LLMs to increase results in a variety of areas, such as ransomware detection [29] and IoT malware detection using information from network packets [15]. These approaches differ from ours in that we analyze packet streams collected directly from a home router ecosystem and generate embeddings using a task-specific packet abstraction language, rather than relying on generic LLM tokenization or pre-trained embedding spaces. There have also been efforts to detect Android malware using LLMs, which attempts to model semantic dependencies within Android application packages (APKs) [4].

Lastly, there has been recent work on the detection of malware on Linux devices using representative call systems from the device and a range of LLMs, including BERT, GPT-2, and Mistral [21]. This body of work overlaps with our studies, although it has some key differences. One is that their work uses pre-trained models with a classification layer on top [21] for model fine-tuning, whereas our models are trained using run-time system call or network packet data. Additionally, their models primarily perform supervised binary classification, whereas our approach is unsupervised anomaly detection, which is better aligned with detecting zero-day or stealthy attacks.

2.3 Contrastive Learning

Contrastive learning has been applied to several domains, such as anomaly detection in graphs [10] and images [9]. It has also been used recently in conjunction with LLMs for medical research [25], code authorship [26], and training LLMs to respond in a certain way to align with the intent of a user [6]. These are just a few examples of widely divergent fields that are all harnessing the power of transformer-based models using contrastive learning. Similarly, contrastive methods with augmented or perturbed negatives have been used in a wide array of problem domains, such as facial recognition [8] and natural language processing [12], although to our knowledge, this type of learning has not yet been applied to the detection of behavioral anomalies in router-level system call or packet data.

There are many variants of contrastive loss, such as mean-shifted contrastive loss, introduced by Reiss et al. [18], as well as triplet loss [20]. Triplet loss is a popular loss function for machine learning models in a wide range of model applications, such as computer vision [20], and aims to learn meaningful data representations by comparing the distances of the three triplet vectors [7]. Our triplet loss framework is similar to the one used by Schroff et al., in which the authors used a triplet mining strategy to align matching and non-matching faces [20]. In our setting, the same structure is used to learn representations of benign router behavior, where negatives are drawn from other benign sequences and lightly perturbed to create harder contrastive examples for anomaly detection.

3 Methodology & Experimental Setup

3.1 Network Ecosystem

The IoTOwl testbed is a small home-network ecosystem built around a consumer-grade router configured to serve as the gateway for all connected devices. All system call and packet telemetry used in this study originate from this device, while a lightweight cloud dashboard is used only to visualize activity. The ecosystem includes six heterogeneous IoT and user devices connected through three common home-network protocols (WiFi, Bluetooth Low Energy, and Zigbee). The devices of our testbed include a PurpleAir PA-II air quality sensor (WiFi), a BerryMed pulse oximeter (BLE), a Philips Hue light bulb (Zigbee), a Google Home (WiFi), a smartphone (WiFi) and a laptop (WiFi).

Each of the sensors first connects to the router using its respective protocol, and then the server runs several programs that get the current sensor readings from the devices and pushes them to a Grafana dashboard. The combination of gateway programs that receive sensor readings and the AWS Grafana server encompasses an IoT ecosystem called IoTOwl, first introduced in previous work [2]

3.2 SkyShark

To collect high-fidelity system call and network telemetry from the router, we use the Extended Berkeley Packet Filter (eBPF) subsystem [11] in a tool called SkyShark, first developed in previous work [2]. eBPF enables dynamic instrumentation of the Linux kernel and allows real-time monitoring of system call and network events without modifying system binaries. The SkyShark tool is designed to be lightweight, yet effective, which is imperative for sensing on resource-constrained devices such as routers. To this end, SkyShark has been tested on several resource-constrained Linux devices, including the consumer-grade GL.iNet GL-MT6000, and did not inhibit the normal functionality of the router.

While the previous work [2] focused exclusively on system call collection, we extend SkyShark to gather network packet data as well. Figure 1 shows the eBPF attachment points used in the kernel's networking stack. Because attribution of network activity to userspace processes is essential for our anomaly detection approach, we instrument the socket layer to obtain process identifiers (PIDs) for packets generated by userspace applications. To complement the socket-layer view, which does not expose lower-layer protocol fields, we also instrument the Traffic Control (TC) layer to capture full Layer 2–4 header information for all ingress and egress packets.

3.3 System Call Collection

As noted above, system call traces are collected using the custom-built SkyShark tool. Alternative userspace tools such as `ftrace` excel as general-purpose utilities for system administrators who perform broad performance analysis and debugging [22]. Our sensor

attaches selectively to syscall tracepoints or kprobes and extracts only the arguments, PIDs, and metadata needed for tokenization and embedding. It avoids the broader event sets such as scheduler and interrupts that `ftrace` often enables by default [5]. To this end, the SkyShark tool is designed to be lightweight, yet effective, which is imperative for sensing on resource-constrained devices such as routers.

This targeted design minimizes data volume from the start. Our eBPF programs apply in-kernel filtering and aggregation before events reach userspace ring buffers for processing. This differs from `ftrace` sessions that may require post-collection processing even when filtered [3]. For our AI platform, this efficiency supports high-fidelity capture of syscall sequences under load, where `ftrace`'s per-CPU buffers, designed to protect system stability by throttling writes, can drop events during bursts and explicitly report these losses via `trace` and `dropped` counters if not finely tuned for our specific use case [22].

eBPF's programmable nature further aligns with our needs. It streams filtered events via low-overhead ring buffers to userspace and preserves timing and ordering, which are critical for behavioral malware anomaly detection [5]. Each system call is mapped to a token using the sys2vec language [2], which groups calls into semantic categories (e.g., file access, networking, and process control). The resulting tokenized sequences form the input to the embedding and contrastive learning pipeline described in the following subsections. Adapting `ftrace` for ML-specific tokenization like sys2vec categories would demand extra scripting and lacks eBPF's native integration for custom schema output [3].

System calls are recorded during both benign execution and controlled malware runs. For offline evaluation, we retain only calls whose PIDs correspond to malware processes so that anomaly scores reflect only malware behavior. In deployment scenarios, where PID filtering is not available, we introduce an exponential moving average (EMA) to smooth anomaly scores over time. This

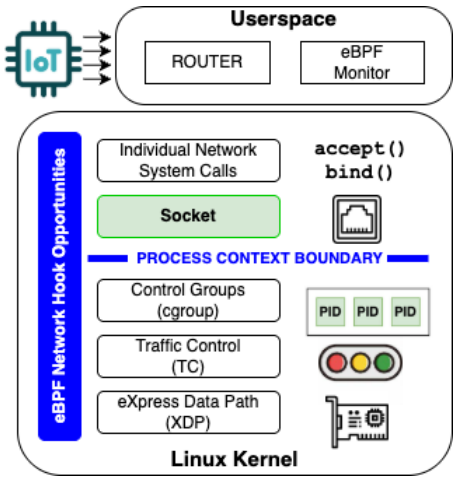

Figure 1: Linux kernel eBPF attachment points for system call and network observability. We intercept data at both the Socket and TC layers to capture complete network context with process attribution.

architecture leverages eBPF's strengths for our specialized pipeline and delivers cleaner, lower-overhead data than repurposing a general tracer like `ftrace` [5, 22].

3.4 Network Collection & Representation

Network packets are similarly collected using eBPF programs attached at both the socket layer, where attribution to userspace processes is preserved, and the traffic-control (TC) layer for low-level packet inspection of all system-transiting traffic. This dual-instrumentation approach captures process-specific network behavior alongside ambient router traffic, complementing our syscall traces with complete communication context.

Network packets are transformed into a discrete "packet language" using a lightweight abstraction scheme. Instead of embedding raw headers, we extract selected fields (e.g., IP addresses, port ranges, direction, and protocol identifiers) and map them into categorical groups such as `DirectionInbound` or `ProtoTcp`. These categorical components are kept as separate tokens within one packet and each has its own vector representation.

Network traffic provides a complementary behavioral signal that system calls alone cannot fully capture. Although network activity is ultimately generated through system calls such as `socket`, the syscall stream offers only a noisy and fragmented view of higher-level communication patterns. Key characteristics of malware—such as destination address regularity, port-selection strategies, burstiness of outbound flows, and repeated connection attempts—are directly observable at the packet level but are difficult to infer reliably from low-level syscall sequences. For this reason, packet abstractions provide a more stable and discriminative representation of network-intensive malware, especially at low false-positive operating points where syscall-only models struggle.

This abstraction reduces noise from high-entropy header fields, improves generalization across devices, and enables packet traces to be treated as sequences analogous to system calls. Each unique packet abstraction token is then embedded using net2vec, a Word2Vec-style embedding model tailored to the packet vocabulary. The resulting embeddings provide continuous vector representations suitable for transformer-based sequence modeling.

Like many types of device communication, packet traffic can be reduced to a small set of vocabulary words, each with a distinct meaning. Since all packets have a similar structure, the key to differentiating them by their function is the effective abstraction of the packet fields into meaningful categories. For example, IP addresses are often ephemeral and therefore not helpful for training a generalizable model. Similarly, granular values such as the exact byte size of a packet create a sparse vocabulary that is not conducive to model training.

After preprocessing, the protocol, source IP, source port, destination IP, destination port, size in bytes, calling process and packet direction fields are left, which are then bucketed into more general groups. IP addresses are grouped based on their address class into "Public", "Private", "Loopback", "LinkLocal", and "Multicast" (plus "Invalid" for malformed values). For example, a source IP may be transformed into "SrcIpPrivate" for the packet above. Ports are abstracted in two ways. For destination ports, if they are in a subset of IANA-assigned ports and appear in the benign training data at least

50 times, the representation given to the port is the specific name. The subset includes SSH (22), DNS (53), HTTP (80), HTTPS (443), IMAPS (993), MQTT (1883), HTTP-Alt (8080), and MQTTS (8883), reflecting common router traffic (SSH, DNS, HTTP, HTTPS) as well as protocols specific to the IoT deployment (MQTT, MQTTS). For example, if the port observed is 443, the final representation is "DstPortHttps." The second way is by Linux port range: ports from 0 to 1023 are categorized as well-known, ports from 1024 to 49151 as registered, and ports from 49152 to 65535 as dynamic. This is used for source ports since they are often ephemeral, as well as destination ports not in the list above or not observed in the benign training data at least 50 times. The calling process field is either ProcUnknown, if it was not in the raw data, Proc<name>with the specific name of the process if the name appeared in the benign training data at least 50 times, or ProcOther, if not.

From these abstractions, we train net2vec using the Word2Vec package available from Gensim [17]. Although the vocabulary size of 34 is small compared to natural languages, this vocabulary is sufficient for the number of packet-behavior patterns present in the router environment.

As part of our ablation study, we also evaluated fixed random embeddings for separate comparisons with sys2vec and net2vec. It was found that sys2vec yielded more consistent results with less variance compared to the fixed random embeddings. However, for the network-based pipeline, the fixed random embedding performance matched the net2vec performance. However, in order to evaluate the system call and network traffic pipelines as closely as possible, we describe only the net2vec pipeline for the duration of the work.

3.5 Malware Patterns

We implement eight malware behaviors following the specifications of the prior work [2] to enable direct comparison. Each malware sample runs for five minutes. To capture the malware PIDs for evaluation, we use standard tools such as `pstree`. The number of observations per malware type is shown in Table 1. We group malware patterns into two categories: (1) OS-focused behaviors that leave no network footprint, and (2) network-focused behaviors that can be detected using both system calls and packet abstractions.

The first OS-focused malware is **traverse**, which walks the filesystem and issues a `stat` call for each file, emulating reconnaissance malware seeking sensitive information. The second is **encrypt**, which similarly traverses the filesystem but encrypts and decrypts each file using gpg, reflecting common ransomware behaviors. **rename** walks the filesystem and repeatedly renames files, mimicking malware attempting to hide or stage files. **compile** compiles small C programs and deletes them, representing malware that compiles payloads after landing on a target device.

The first network-focused malware is **download**, which fetches code from a `git` repository, representing malware attempting to obtain remote payloads. The next is **lateral**, which searches for credentials in locations such as `/etc/shadow` and attempts `ssh` connections to internal hosts, combining reconnaissance and lateral-movement behavior. **combo** merges multiple behaviors: downloading code, compiling code, and traversing and encrypting the filesystem, representing a multi-stage infection chain. Finally, the **APT** is a client/server application that exfiltrates data to an attacker-controlled server while rate-limiting bandwidth and sleeping between bursts to remain stealthy. In our setup, it transmits for 60 seconds and sleeps for 10 seconds with a 100 kB/s rate limit, still producing substantial network activity (Table 1).

3.6 Dataset Characteristics

We evaluate our models using system call and network packet traces collected from the IoTOwl ecosystem. The benign data consist of four separate traces: one 75-minute capture used for model training, and three 15-minute captures used exclusively for evaluation. The system call portion of `benign-75min` is truncated to 1,000,000 calls to reduce training time while preserving behavioral diversity, while all packet abstractions observed during this period are included to build the full net2vec vocabulary. The truncation is a computational compromise rather than an assumption about sufficiency. The three additional benign traces (`benign-15min`, `benign2-15min`, and `benign3-15min`) are used for evaluation. Using multiple benign evaluation sets allows us to measure the generalizability of the detector across independent, separately collected benign periods rather than relying on a single benign baseline. Each malware sample is executed for 5 minutes. During the evaluation, we retain only the system calls and packets associated with the malware process identifiers (PIDs), which isolate the behavioral footprint of the malware from background traffic. This results in varying observation counts between malware types depending on their level of activity (Table 1). Several malware samples produce small numbers of observations (e.g., a few hundred packets), such as download, which only performs an isolated call to `git` before exiting. Consequently, extreme detection values (e.g., from 0% to 100%) should be interpreted as single-case results rather than as statistically stable estimates. Expanding the number and diversity of malware samples is an important direction for future work.

Table 1 summarizes the total number of system call and packet observations used for training and evaluation. The separation between training (one long benign trace) and testing (three independent benign traces plus malware traces) ensures that the model is evaluated on behavior it has never seen before, matching the anomaly-detection objective and avoiding any form of leakage across data splits.

Table 1: Number of data observations.

Dataset	Syscall Count	Packet Count
benign-75min	1,000,000	1,434,100
benign-15min	2,803,508	342,989
benign2-15min	2,663,153	228,168
benign3-15min	2,733,216	254,723
lateral-5min	4,167	1,406
apt6010-5min	54,863	24,291
download-5min	2,191	180
combo-5min	99,952	1,221
rename-5min	29,443	—
traverse-5min	40,553	—
encrypt-5min	147,564	—
compile-5min	704	—

3.7 Triplet Construction

System call sequences are segmented into fixed-length windows of 500 tokens, while packet sequences are segmented into windows of 70 packets with each containing eight tokens. These window sizes were chosen empirically to provide sufficient temporal and contextual coverage without inflating sequence length unnecessarily.

For all offline experiments, the malware data contain only system calls or network packets associated with malicious PIDs. This ensures that only malware-generated data are used in the evaluation, as mixing of benign and malware data yields imprecise results and the goal is to measure how far a malware process deviates from the learned benign profile. It does not assume that PID information is available during deployment (this is addressed separately by our online detector).

From these sequences we construct triplets consisting of an anchor, a positive, and a negative example drawn from benign data only. The anchor is a benign window, the positive is the next sequential benign window, and the negative is a randomly selected benign window from a later point in the same PID stream. This structure encourages the model to learn stable representations of benign behavior by bringing temporally adjacent windows closer together while pushing unrelated windows further apart.

3.8 Contrastive Augmented Learning

To increase the difficulty of the contrastive task, we apply a lightweight mutation to each negative sample: a small fraction of its tokens is replaced with randomly selected tokens drawn from the overall training vocabulary. This is not intended as an adversarial attack, but rather as a controlled augmentation that produces harder negative examples.

The motivation for mutating negative samples is to generate negatives that remain close to the benign manifold while differing in semantically meaningful ways. Standard contrastive learning draws negative windows from unrelated parts of the benign trace, but we find that some mutation to the randomly-selected negative sentence produces more stable embeddings and improves anomaly detection performance at low false positive rates relative to standard contrastive learning without mutation.

3.9 calBERT Model Architecture

We train two BERT-based models from scratch: one for system calls using sys2vec embeddings and one for network packets using our net2vec embeddings. The architectures are parallel so that differences in performance arise only from the underlying data types rather than differences in model capacity or training procedure. Only benign data are used for training, consistent with the anomaly-detection objective.

For the system call model, we follow the contrastive augmented learning framework described above. For the contrastBERT replication, we adopt a hard-negative mining strategy: for each anchor–positive pair, we sample 50 candidate windows and select the candidate with maximum embedding distance as the negative sample.

For mutation rates $m \in \{0.1, 0.2\}$, we select a future window as the negative sample and mutate m proportion of its tokens by replacing them with randomly selected system calls. This produces

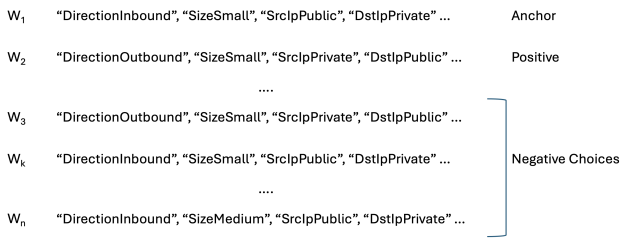

Figure 2: Example of how the negative window is selected from future windows relative to the current anchor and positive sample.

harder negative examples and allows us to evaluate the impact of controlled augmentation on model performance.

The network-based BERT model is trained similarly, using the packet abstraction language and the net2vec embeddings. For each anchor window, the subsequent window is used as the positive, and the negative is chosen as a random future window. The negative window is then mutated by replacing a fraction of the packet tokens with other packet abstractions observed in the training data. We sweep mutation rates of 0, 0.1, and 0.2 to compare their effects, where the 0% mutation is simply a randomly-selected negative sample without any mutation. In practice, 10% mutation provides the most stable performance.

Both models use the `BertModel` class from HuggingFace Transformers [24] as part of the model pipeline. A fully connected projection layer maps the 64-dimensional sys2vec or net2vec embeddings into the 768-dimensional BERT embedding space, followed by LayerNorm and positional embeddings. After the BERT encoder, the representations are mean-pooled and ℓ_2-normalized. Cosine similarities between anchors, positives, and negatives are passed into the margin-based contrastive loss from previous work [2]. The margin is set to 0.1 for both models. The batch size is set to 32 for both models and a learning rate of 1×10^{-6} is used. Additionally, to help with overfitting, we use $p_{\text{dropout}} = 0.1$ and weight decay of $\lambda = 0.1$. The full architecture is shown in Figure 3.

$$\mathcal{L} = \mathbb{E}\left[\max\left(0,\ -\cos(\mathbf{a}, \mathbf{p}) + \cos(\mathbf{a}, \mathbf{n}) + \text{margin}\right)\right] \tag{1}$$

3.10 Anomaly Scoring with Isolation Forest

Our BERT models are representation learners: during inference, they produce fixed-dimensional embeddings for each 70-packet (560-token) window but do not directly output anomaly labels. To convert these embeddings into anomaly scores, we use an Isolation Forest (IF), a standard unsupervised anomaly detector widely used in representation-learning pipelines. The IF is trained solely on embeddings from benign data. In contrast to the prior work [2], we found that using the raw benign embeddings yields more stable performance and therefore train the IF directly on the embedding vectors.

Before fitting the isolation forest, we normalize and apply a StandardScaler, available from Scikit-learn, [16] fitted on the concatenated benign embeddings. The same scaler is applied to the

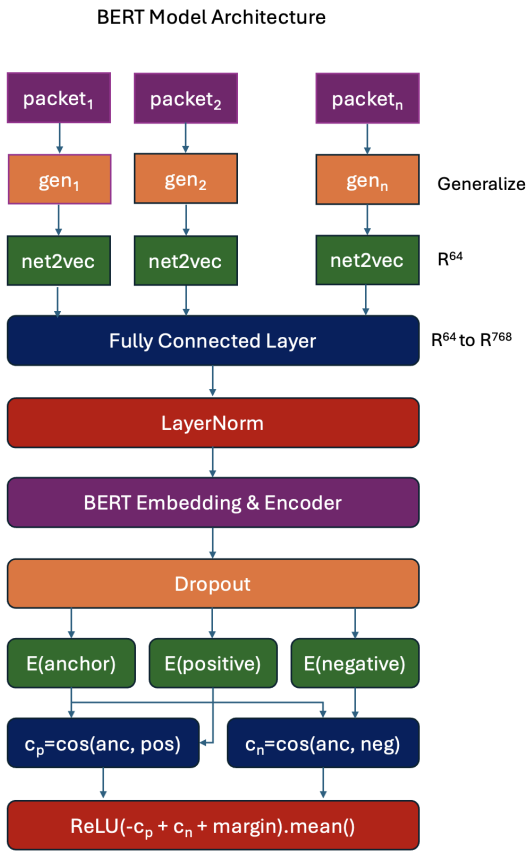

Figure 3: BERT architecture for both models. This figure shows the pipeline for packets, which use the generalization layer plus net2vec embeddings. For the system call architecture, the input is system calls, there is no generalization layer, and net2vec is replaced with sys2vec.

three benign datasets as well as to the malware dataset during testing. Following that, the isolation forest implementation from Scikit-learn [16] is used with `random_state=42`, `n_estimators=200`, and `max_samples=min(1024, n)`, where n is the number of benign training embeddings. These settings provide better performance than the default settings of 100 trees and 256 samples.

The contamination parameter of the IF corresponds to the expected fraction of anomalies (i.e., the target false positive rate), allowing us to evaluate performance at multiple operating points. We report results at contamination levels (false positive rates) 0.001, 0.005, and 0.015. Reporting results at the 0.001 contamination level is shown to properly illustrate the level at which the benefits of augmented learning and the addition of the network-based pipeline is most visible. Table 1 summarizes the sizes of datasets for both training and testing the model.[1]

[1]https://github.com/anonymous-researcher-521/iotowl-data

3.11 Online Anomaly Detection

In a real deployment, unlike offline analysis, PID information is not assumed and is not used. The router observes a single mixed stream of system-level events and network packets produced by all processes. For this setting, the model operates directly on this interleaved stream: events are embedded, windows are formed sequentially over time, and anomaly scores are produced without any reference to PIDs. The unfiltered mixed-stream online processing for deployment ensures that the offline results measure intrinsic detector quality while the online pipeline reflects the practical operating mode of a real router. To support this deployment scenario, we implement an online anomaly detection pipeline that computes anomaly scores over a live stream of embedded tokens. Raw scores are smoothed using an exponential moving average (EMA), which reduces sensitivity to short-lived benign bursts and stabilizes the decision boundary.

3.12 Limitations

Although the approach yields strong results, there are several limitations that affect the statistical strength of the findings. First, the evaluation is based on a small set of scripted malware behaviors running for short durations, and several samples generate only limited numbers of system calls or packets. As a result, observed detection rates—especially extreme values such as 0% or 100%—should be interpreted as case-study outcomes rather than precise population estimates. We do not perform confidence intervals, hypothesis testing, or multiple runs with different random seeds, which limits the statistical rigor of comparisons. Second, the benign training distribution reflects a single router ecosystem with a small number of devices. While this provides a realistic testbed, it restricts the diversity of benign behavior and may not capture households with significantly different patterns. The truncation of the 13.8M-call benign trace to 1M calls further reduces diversity, although it was necessary for computational feasibility. Third, the packet abstraction language simplifies fine-grained header fields into broad categories, improving generalization but discarding subtle information that may be discriminative for advanced threats. Finally, extremely stealthy or long-lived malware may evade both system call–based and packet-based detectors, and EMA smoothing in the online detector introduces a tradeoff between stability and responsiveness. These limitations highlight opportunities for future work in expanding the diversity of the dataset, adding statistical confidence measures, and exploring more expressive packet representations.

4 Anomaly Detection Results

4.1 Offline Analysis

To evaluate our research contributions, we examine three separate, yet related, problems in detail. The first is how the introduction of augmented learning in model training can improve system call-based malware detection. These results are shown in Table 2. The second contribution is the addition of network traffic data to augment the system call data, and these results are summarized in Table 3. The final contribution is the online anomaly detector that validates the offline work in a real-world setting, which is explained in the next section.

Table 2: System call detection rates (%) for sys2vec embeddings with contrastive augmentation. The conBERT columns correspond to results from the previous contrastBERT work by the authors [2]. For the other columns, the mutation rates are listed above each column block; bold indicates the best result for each malware under each contamination level. The contamination sweep is extended to 0.001 (a lower FPR target than the previous work) to show where mutation augmentation provides the largest benefit, since detection saturates above 0.005 for most malware patterns. As such, the conBERT entries are only applicable at the 0.005 and 0.015 contamination rates and are left blank for 0.001.

Malware	0.001				0.005				0.015			
	conBERT	0.0	0.1	0.2	conBERT	0.0	0.1	0.2	conBERT	0.0	0.1	0.2
traverse	–	0.00	0.00	0.00	0.00	0.00	**7.41**	0.00	0.06	**100.00**	**100.00**	**100.00**
encrypt	–	0.00	0.00	0.00	11.64	**99.31**	98.96	79.17	**100.00**	**100.00**	**100.00**	**100.00**
rename	–	0.00	8.62	**13.79**	48.27	98.28	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**
compile	–	0.00	0.00	**100.00**	0.00	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**
download	–	0.00	**50.00**	**50.00**	0.00	**100.00**	75.00	75.00	**100.00**	**100.00**	**100.00**	**100.00**
lateral	–	0.00	0.00	0.00	**33.05**	25.00	25.00	25.00	99.44	**100.00**	**100.00**	**100.00**
combo	–	0.00	**0.51**	**0.51**	0.00	19.80	**70.56**	6.60	1.79	**98.48**	**98.48**	**98.48**
apt6010	–	0.00	0.00	0.00	0.00	**0.92**	0.00	0.00	0.00	**100.00**	**100.00**	**100.00**

Table 3: Detection rates (%) for system call-based (Sys) and network-based (Net) detectors for network-focused malware. System call values correspond to Mut=0.1 in Table 2. Network values report the saturated detection rate; mutation rates 0.0, 0.1, and 0.2 all achieve 100% detection at every contamination setting, so we report a single Net column per contamination for readability. Best value per malware per contamination rate is in bold.

Malware	0.001		0.005		0.015	
	Sys	Net	Sys	Net	Sys	Net
download	50.00	**100.00**	75.00	**100.00**	**100.00**	**100.00**
lateral	0.00	**100.00**	25.00	**100.00**	**100.00**	**100.00**
combo	0.51	**100.00**	70.56	**100.00**	98.48	**100.00**
apt6010	0.00	**100.00**	0.00	**100.00**	**100.00**	**100.00**

Overall, the experimental results show that augmented learning improves the results for both the system call and network detectors. Similarly, using network packets as input to the model drastically increases the detection rate at the lowest false positive rate shown in three out of the four network-focused malware cases.

Table 2 describes our system call-based detection results. We show three contamination rates in the isolation forest, where the contamination parameter specifies the expected proportion of anomalies in the data and, therefore, directly controls the false positive rate. The first column restates the results from prior work [2]. This column is only applicable to the 0.005 and 0.015 contamination rates since the previous work does not report results at the 0.001 contamination rate. The second column shows our replication using the improved eBPF-based sensor, which offers high-fidelity system-call collection and reduces drop rates under load, leading to better performance even without augmentation. It also introduces the scaler and larger isolation forest definition. The third column shows the results using augmented learning with a mutation rate of 10%. This shows a noticeable improvement over non-mutated learning at the 0.001 contamination rate for the download malware. The last column is the same as the previous column except that the mutation rate used is 20%. This further improves detection at the 0.001 contamination rate for the compile malware.

Table 3 shows results for the four network-focused malware. Since the results across all the mutation rates are saturated at 100%, we show only one condensed network column for each contamination rate. The improvement from using network input data over system call input data is most visible at the 0.001 and 0.005 contamination rates, which in some cases improves detection from 0% to 100%.

4.2 Online Analysis

In the offline evaluation setting, system calls and packets can be filtered by process identifier (PID), enabling accurate anomaly scoring. However, a real deployment on a home router cannot rely on PID filtering, since packet streams and system activity must be analyzed continuously without malware behavior isolation. To support this setting, we implement an online anomaly detection pipeline that computes anomaly scores over a live stream of embedded tokens.

Raw scores from the Isolation Forest are smoothed using an exponential moving average (EMA), which reduces sensitivity to short-lived bursts of benign activity and stabilizes the decision boundary in the absence of PID separation. The EMA is defined in Equation 2, where c_t is the current score and v_{t-1} is the previous smoothed value:

$$v_t = \alpha \cdot c_t + (1 - \alpha) \cdot v_{t-1}. \tag{2}$$

The EMA attenuates brief benign spikes—common in IoT polling and background router activity—while still reacting to persistent deviations indicative of malware. The α parameter is set to be 0.7

Table 4: Time-to-detection (TTD) results for syscall and network detectors across contamination rates. The number of times in which the malware was detected in the 10 trials is shown in brackets. All times in seconds. "–" indicates the detector channel is not applicable for that malware pattern.

	0.001		0.005		0.015	
Malware	**Syscall**	**Network**	**Syscall**	**Network**	**Syscall**	**Network**
Traverse	[0/10]	–	0.140 ± 0.020 [10/10]	–	0.126 ± 0.043 [10/10]	–
Encrypt	[0/10]	–	0.127 ± 0.033 [10/10]	–	0.136 ± 0.033 [10/10]	–
Rename	0.313 ± 0.038 [10/10]	–	0.742 ± 1.740 [10/10]	–	0.156 ± 0.040 [10/10]	–
Compile	[0/10]	–	0.121 ± 0.023 [10/10]	–	0.126 ± 0.044 [10/10]	–
Download	[0/10]	8.944 ± 0.128 [5/10]	0.126 ± 0.031 [10/10]	8.885 ± 0.089 [10/10]	0.114 ± 0.027 [10/10]	9.243 ± 0.650 [8/10]
Lateral	[0/10]	2.430 ± 0.930 [10/10]	0.116 ± 0.028 [10/10]	1.750 ± 0.549 [10/10]	0.139 ± 0.086 [10/10]	1.485 ± 0.827 [10/10]
Combo	[0/10]	9.123 ± 0.595 [4/10]	0.130 ± 0.079 [10/10]	9.043 ± 0.360 [10/10]	0.140 ± 0.048 [10/10]	9.320 ± 0.644 [10/10]
APT	[0/10]	26.827 ± 38.462 [10/10]	0.128 ± 0.054 [10/10]	2.127 ± 1.732 [10/10]	0.147 ± 0.034 [10/10]	0.983 ± 0.776 [10/10]
FPR	0.0000	0.0000	0.0000	0.0000	0.0000	0.0003

to balance the current observation while not giving one observation too much weight. To further reduce false positives without sacrificing speed of detection, we add a short debouncing period of two consecutive anomalies before flagging an anomaly. This ensures that one-off false positives that slip through the smoothing algorithm do not set off an alarm, while ensuring anomalies are detected as quickly as possible. A final anomaly decision is made by comparing the smoothed score to a threshold set as the $(100 \times \text{contamination})^{\text{th}}$ percentile of benign training scores, providing a direct mapping between a chosen false positive budget and the operational boundary. This online evaluation complements the PID-filtered offline results by demonstrating performance in a realistic deployment scenario where mixed benign and malicious activity must be processed continuously.

The main metric used to evaluate online anomaly detection is time-to-detection (TTD). The TTD and the false positive rate are the most important statistics to analyze in online anomaly detection because anomaly detection must be quick and correct. Our results in Table 4 show that all malware patterns can be detected in less than a second after initial malware infection at the 0.005 and 0.015 false positive rates using system calls, and all of the network-focused malware can be detected at every false positive rate using the network-based detector. The usefulness of the network pipeline is most pronounced at the 0.001 false positive rate, in which none of the network-focused malware can be detected using the system call pipeline.

5 Future Work

An area to explore in future work is incorporating a BLE sensor and its data. Network packet data are, in essence, subsets of system call data, since network traffic is seen through system calls such as `socket`. However, the results in this research show that while that may theoretically be the case, in practice there is too much noise to pinpoint those actions as clearly with system calls as is possible using network packet data. Another area to explore is to try more recent transformer models. BERT remains a remarkable all-purpose model that is relatively accessible for researchers without the resources of large AI companies, but the vanilla BERT model came out several years ago, which is a long time in today's accelerated AI research timeline. In addition, though the vanilla BERT model yields excellent results, we would like to implement even stealthier malware and more complex ecosystem configurations to test the limits of the model. To this end, alternative models to try are XLNet [27] and ModernBERT [23].

6 Conclusion

This work focuses on advancing the state of the art in behavioral anomaly detection for edge devices. We introduced augmented learning during the training process to show how this improves system call-based malware detection. We then applied a similar pipeline to network traffic packet abstractions, which yields better detection for network-intensive malware at low false positive rates. We also provided an online anomaly detector suitable for real-world use to validate our approach.

The system call and network packet data are collected from a router that services a realistic home network consisting of several devices using a variety of communication protocols. Once the packets are collected from the router, they are generalized to packet abstractions. These abstractions are then embedded into 64-length embeddings by a Word2vec-like model we call net2vec. We also tried using fixed random embeddings as part of an ablation study and found that they yielded identical results, as noted above. After creating vector embeddings for each packet word in the observed vocabulary, we process the data into sentences of length 500 for syscalls and 70 for network packets (including eight tokens for each packet), which are then split into triplets. Each triplet contains an anchor, a positive example (similar to the anchor), and a negative example (designed to be dissimilar to the anchor). The random negative is then mutated by changing a portion of the packets in the negative sentence to different packet configurations seen in the training data. Using these triplets and the custom triplet loss function, the BERT model learns to keep the anchor and positive similarity high while attempting to reduce the similarity between the anchor and the negative example. The BERT model is then evaluated by fitting an isolation forest on three separate benign datasets and detecting outliers in several types of malware patterns.

The experimental results show that using augmented learning as part of the training process increases system call-based detection at low false positive rates. Similarly, a model trained on network data works even better for network-focused malware at low false positive

rates, which is expected given the more pronounced footprint of network-focused malware in the network traffic.

In this work, three main research contributions are presented. The first is the introduction of augmented learning to the BERT training, which builds on the contrastBERT results and yields a meaningful improvement over that work at the lowest false positive rates. The second is the introduction of a new network packet abstraction language that provides a pipeline similar to that for network packets as that for system calls. We show that the network-based pipeline improves the detection of every network-focused malware examined at the two lowest false positive rates examined. The third is the fully online detector that validates this research for real-world use. The online detector is often able to find malware in less than a second at the two higher false positive rates examined, and still yields useful results for the 0.001 target false positive rate using the network traffic pipeline. Given these results, the optimal setup for such an anomaly detector would use both system calls and network traffic data as two separate pipelines that complement each other in terms of speed and detection coverage. This research improves the state of behavioral anomaly detection by providing a deployable online detection architecture for real-world networks built on an innovative machine learning model custom-trained for this problem space.

Ethical Considerations

The goal of this work is to improve the security of edge devices. The data was collected in a closed ecosystem using only malware that does not propagate or infect any unintended nodes by design. No malicious code used in this work is shared publicly and no personally identifiable data were used.

Acknowledgments

This research was funded by the Auerbach Berger Chair of Cybersecurity held by Spiros Mancoridis.

References

[1] Mennatallah Amer, Markus Goldstein, and Slim Abdennadher. 2013. Enhancing one-class support vector machines for unsupervised anomaly detection. In *Proceedings of the ACM SIGKDD Workshop on Outlier Detection and Description* (Chicago, Illinois) *(ODD '13)*. Association for Computing Machinery, New York, NY, USA, 8–15. doi:10.1145/2500853.2500857

[2] John Carter, Spiros Mancoridis, Pavlos Protopapas, and Brian Mitchell. 2025. contrastBERT: Behavioral Anomaly Detection for Malware using Contrastive Learning. In *Proceedings of the Conference on Game Theory and AI for Security (GameSec '25))*.

[3] Cyril Cassagnes, Lucian Trestioreanu, Clement Joly, and Radu State. 2020. The Rise of eBPF for Non-Intrusive Performance Monitoring. In *2020 IEEE/IFIP Network Operations and Management Symposium (NOMS)*. 1–7. doi:10.1109/NOMS47738.2020.9110434 Budapest, Hungary.

[4] Ruirui Feng, Hui Chen, Shuo Wang, Md Monjurul Karim, and Qingshan Jiang. 2025. LLM-MalDetect: A Large Language Model-Based Method for Android Malware Detection. *IEEE Access* 13 (2025), 81347–81364. doi:10.1109/ACCESS.2025.3565526

[5] William Findlay. 2020. *Security Applications of Extended BPF Under the Linux Kernel*. COMP5900T OS Security Research Paper. Carleton University, School of Computer Science. https://www.cisl.carleton.ca/~will/written/findlay20bpfsec.pdf

[6] Xiang Gao and Kamalika Das. 2024. Customizing Language Model Responses with Contrastive In-Context Learning. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 16 (Mar. 2024), 18039–18046. doi:10.1609/aaai.v38i16.29760

[7] Elad Hoffer and Nir Ailon. 2018. Deep metric learning using Triplet network. arXiv:1412.6622 [cs.LG] https://arxiv.org/abs/1412.6622

[8] Daeha Kim and Byung Cheol Song. 2021. Contrastive Adversarial Learning for Person Independent Facial Emotion Recognition. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 7 (May 2021), 5948–5956. doi:10.1609/aaai.v35i7.16743

[9] HyunGi Kim, Siwon Kim, Seonwoo Min, and Byunghan Lee. 2024. Contrastive Time-Series Anomaly Detection. *IEEE Transactions on Knowledge and Data Engineering* 36, 10 (2024), 5053–5065. doi:10.1109/TKDE.2023.3335317

[10] Xuexiong Luo, Jia Wu, Jian Yang, Shan Xue, Hao Peng, Chuan Zhou, Hongyang Chen, Zhao Li, and Quan Z Sheng. 2022. Deep graph level anomaly detection with contrastive learning. *Scientific Reports* 12, 1 (2022), 19867.

[11] Sebastiano Miano, Xiaoqi Chen, Ran Ben Basat, and Gianni Antichi. 2023. Fast In-kernel Traffic Sketching in eBPF. *ACM SIGCOMM Computer Communication Review* 53, 1 (2023).

[12] Deshui Miao, Jiaqi Zhang, Wenbo Xie, Jian Song, Xin Li, Lijuan Jia, and Ning Guo. 2021. Simple Contrastive Representation Adversarial Learning for NLP Tasks. arXiv:2111.13301 [cs.CL] https://arxiv.org/abs/2111.13301

[13] Ali Bou Nassif, Manar Abu Talib, Qassim Nasir, and Fatima Mohamad Dakalbab. 2021. Machine Learning for Anomaly Detection: A Systematic Review. *IEEE Access* 9 (2021), 78658–78700. doi:10.1109/ACCESS.2021.3083060

[14] Dimosthenis Natsos and Andreas L. Symeonidis. 2025. Transformer-based malware detection using process resource utilization metrics. *Results in Engineering* 25 (2025), 104250. doi:10.1016/j.rineng.2025.104250

[15] Marwan Omar, Hewa Majeed Zangana, Jamal N. Al-Karaki, and Derek Mohammed. 2024. Harnessing LLMs for IoT Malware Detection: A Comparative Analysis of BERT and GPT-2. In *2024 8th International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT)*. 1–6. doi:10.1109/ISMSIT63511.2024.10757249

[16] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[17] Radim Rehurek and Petr Sojka. 2011. Gensim–python framework for vector space modelling. *NLP Centre, Faculty of Informatics, Masaryk University, Brno, Czech Republic* 3, 2 (2011).

[18] Tal Reiss and Yedid Hoshen. 2023. Mean-Shifted Contrastive Loss for Anomaly Detection. *Proceedings of the AAAI Conference on Artificial Intelligence* 37, 2 (Jun. 2023), 2155–2162. doi:10.1609/aaai.v37i2.25309

[19] Laura Rettig, Mourad Khayati, Philippe Cudré-Mauroux, and Michał Piorkówski. 2019. *Online Anomaly Detection over Big Data Streams*. Springer International Publishing, Cham, 289–312. https://doi.org/10.1007/978-3-030-11821-1_16

[20] Florian Schroff, Dmitry Kalenichenko, and James Philbin. 2015. FaceNet: A unified embedding for face recognition and clustering. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 815–823. doi:10.1109/cvpr.2015.7298682

[21] Pedro Miguel Sánchez Sánchez, Alberto Huertas Celdrán, Gérôme Bovet, and Gregorio Martínez Pérez. 2024. Transfer Learning in Pre-Trained Large Language Models for Malware Detection Based on System Calls. In *MILCOM 2024 - 2024 IEEE Military Communications Conference (MILCOM)*. 853–858. doi:10.1109/MILCOM61039.2024.10773857

[22] Simon Volpert, Sascha Winkelhofer, Jörg Domaschka, and Stefan Wesner. 2025. Towards eBPF Overhead Quantification: An Exemplary Comparison of eBPF and SystemTap. In *Companion of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE '25 Companion)* (Toronto, ON, Canada). ACM, New York, NY, USA, 122–129. doi:10.1145/3680256.3721311

[23] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. 2024. Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference. arXiv:2412.13663 [cs.CL] https://arxiv.org/abs/2412.13663

[24] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and Jamie Brew. 2019. HuggingFace's Transformers: State-of-the-art Natural Language Processing. *CoRR* abs/1910.03771 (2019). arXiv:1910.03771 http://arxiv.org/abs/1910.03771

[25] Yuzhou Wu, Jin Zhang, Xuechen Chen, Xin Yao, and Zhigang Chen. 2025. Contrastive learning with large language models for medical code prediction. *Expert Systems with Applications* 277 (2025), 127241. doi:10.1016/j.eswa.2025.127241

[26] Xiaodan Xu, Chao Ni, Xinrong Guo, Shaoxuan Liu, Xiaoya Wang, Kui Liu, and Xiaohu Yang. 2025. Distinguishing LLM-Generated from Human-Written Code by Contrastive Learning. *ACM Trans. Softw. Eng. Methodol.* 34, 4, Article 91 (April 2025), 31 pages. doi:10.1145/3705300

[27] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. 2020. XLNet: Generalized Autoregressive Pretraining for Language Understanding. arXiv:1906.08237 [cs.CL] https://arxiv.org/abs/1906.08237

[28] Yuan Yao, Abhishek Sharma, Leana Golubchik, and Ramesh Govindan. 2010. Online anomaly detection for sensor systems: A simple and efficient approach. *Performance Evaluation* 67, 11 (2010), 1059–1075. https://www.sciencedirect.com/science/article/pii/S016653161000115X Performance 2010.

[29] Ce Zhou, Yilun Liu, Weibin Meng, Shimin Tao, Weinan Tian, Feiyu Yao, Xiaochun Li, Tao Han, Boxing Chen, and Hao Yang. 2025. SRDC: Semantics-based Ransomware Detection and Classification with LLM-assisted Pre-training. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 27 (Apr. 2025), 28566–28574. doi:10.1609/aaai.v39i27.35080